`matplotlib`

Status: not covered by Software Lab-1 practicals **Where it is used:** `backend/analytics.py` (`save_charts` — the only function in the codebase that imports it), reached from `backend/app.py` (`dataset_charts`, `serve_chart`) **Closest lab concept:** Practical 6, pandas and dataframes — pandas decides *what the numbers are*; matplotlib only decides how many pixels they get

---

1. What it is

matplotlib is a plotting library. You hand it numbers, it hands you a picture. Practicals 0-6 never needed one: you printed tables, counted with `value_counts`, filtered with a boolean mask. matplotlib is the first dependency in this project whose output is **not text**, and that single fact causes every strange-looking choice in `save_charts` — the `"Agg"` backend, the mandatory `plt.close()`, and the fact that charts arrive as PNG files on disk instead of as data inside a JSON response.

It is not part of the syllabus labs, so it is not installed:

```
pip install matplotlib
```

It is also by far the heaviest import in the project — noticeably slower than pandas — which is why it is imported *inside* the function that needs it rather than at the top of `analytics.py`. Every other module in this repo puts its imports at the top. `save_charts` is the exception, and section 3.1 explains why.

---

2. Why the project needed it

The brief asks for cohort analytics over real clinical datasets: how the diabetic and non-diabetic patients differ, and which measurements split the two groups most cleanly. That is a shape question, and a shape question wants a picture.

The interesting decision is **where** the picture gets made. Not in the browser. The frontend is a page that talks to Flask over HTTP; it has no Python and no plotting library. So the backend draws the chart and writes it to disk, and the frontend just asks for the file.

Two routes make that work, and both are in `backend/app.py`:

```
@app.get("/api/analytics/<name>/charts")
def dataset_charts(name):
    """Render charts and return the filenames written."""
    valid, failure = dataset_or_404(name)

    if failure:
        return failure

    written = analytics.save_charts(valid)
    return jsonify({"dataset": valid, "charts": written})
```

```
@app.get("/charts/<path:filename>")
def serve_chart(filename):
    """Serve a generated chart PNG."""
    directory = os.path.join(os.getcwd(), "static", "charts")

    if not os.path.exists(os.path.join(directory, filename)):
        return error(f"no chart named {filename}", 404)

    return send_from_directory(directory, filename)
```

So the flow is: ask for charts → `save_charts()` writes PNGs into `static/charts/` and returns their paths → the frontend fetches each path from `/charts/`. Note the careful `os.path.exists` check before `send_from_directory`: a chart is only downloadable if it has actually been rendered, and it has not been

rendered until someone called the first route.

The tradeoff, stated honestly. Server-side rendering costs the server real work — a matplotlib import, a dataframe scan and a PNG encode on every request — and it means the charts are fixed at the moment they were generated. The alternative is plotting in the browser with a JavaScript chart library, which costs no server work and can redraw on hover and zoom. This project chose server-side because it has **one implementation shared everywhere**: the same PNG is used by the API test suite, by the exported report and by the dashboard, and nobody has to keep two plotting code paths in agreement. For a project where the charts are a deliverable rather than an interaction, that is the right trade. For a live-filtering dashboard it would not be.

There is a second, quieter reason, and it is about testing. `tests/test_analytics.py:354` renders real charts and checks the bytes:

```
def test_save_charts_writes_pngs():
    """Charts are written as real, non-empty PNG files."""
    import os as _os

    for name in analytics.passing_datasets():
        written = analytics.save_charts(name)

        assert len(written) >= 2, (name, written)
        assert_json_safe(written, "chart list")

        for path in written:
            assert _os.path.exists(path), path
            assert _os.path.getsize(path) > 1000, (path, _os.path.getsize(path))
            with open(path, "rb") as f:
                assert f.read(4) == b"\x89PNG", f"{path} is not a PNG"
```

`\x89PNG` is the PNG magic number. This test can only exist because the output is a file: a test can open a file, hash its bytes and read its header. It cannot assert anything about a picture a browser painted.

---

3. The eight pieces actually used

Everything below is inside `save_charts`, `backend/analytics.py:387-481`. There are no other matplotlib calls anywhere in the repository.

3.1 `matplotlib.use("Agg")` — required, not optional

```
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
```

matplotlib is not one library but a core plus a set of interchangeable *backends*. The default backend is a GUI one: it wants to open a window and have a human look at it. In a headless process — a Flask server, a test run, a CI job, this project's Docker image — there is no display to open a window on. matplotlib then either raises or, worse, appears to hang while it waits for a window that will never appear.

`"Agg"` is the *Agg*regate renderer: it draws into an in-memory image buffer and never touches a screen. That is exactly what "write a PNG to disk" needs. Setting it is the standard first line of any server-side matplotlib script, and it must come **before** `import matplotlib.pyplot as plt`, because the backend is chosen at import time.

The order of those three lines is not negotiable, and the fact that they sit inside the function rather than at module top is deliberate too. The top of `analytics.py` imports only `json`, `os` and `pandas`, so importing the module in a test costs nothing extra. `matplotlib` is loaded on the first call to `save_charts` and not before.

3.2 `fig, ax = plt.subplots()` — the figure/axes model

```
        fig, ax = plt.subplots(figsize=(7, 4))
```

This is the single most important idea in the library, and the one most tutorials get wrong by teaching you the wrong style first.

A **figure** (`fig`) is the whole picture — the page, the canvas, the thing that gets saved. An **axes** (`ax`) is one rectangular plotting area *inside* it. A figure can hold several axes (a 2×2 grid of subplots, a main plot plus an inset), and every single thing you draw belongs to an axes: the bars, the lines, the title, the axis labels, the legend.

`plt.subplots()` is a convenience that creates one figure and one axes and returns both. The usual mistake is to ignore `ax` and draw on the module instead, so the call is written `plt.hist(...)`. Both draw the same histogram. What this project actually writes is `ax.hist(...)` — every plotting call in `save_charts` starts with `ax.`, bar the two exceptions named in 3.7 and 3.8.

Both draw the same histogram. The difference is ownership. `plt.hist` draws on "whatever axes happens to be current", which is global mutable state — so the code only works if you got the order right, and silently draws on the wrong chart if a previous figure was never closed. `ax.hist` draws on **the axes object you are holding**. When `save_charts` builds three different charts in one call, each with its own `fig` and `ax`, object ownership is the difference between three charts and a mess.

The reason to care for a lab submission: every parameter is visible at the call site. `ax.hist(series, bins=30, alpha=0.6, label=...)` says what it does. `plt.hist(...)` says the same thing, but you cannot tell from reading it which of the three charts in this function it belongs to.

`figsize=(7, 4)` is width and height in inches. At the `dpi=100` used on save, that is a 700×400 pixel PNG. The real files in `static/charts/` confirm it — `pima_diabetes_groupmeans.png` uses `figsize=(9, 4.5)`.

3.3 `ax.hist(...)`, `alpha=0.6` and `ax.legend()`

```
    # Histogram of the strongest separating column, split by target.
    separators = strongest_separators(df)
    focus = separators[0]["column"] if separators else (cols[0] if cols else None)

    if focus:
        fig, ax = plt.subplots(figsize=(7, 4))

        if target is not None:
            for label, group in df.groupby(target):
                ax.hist(group[focus].dropna(), bins=30, alpha=0.6,
                        label=f"{target}={label}")
            ax.legend()
        else:
            ax.hist(df[focus].dropna(), bins=30)

        ax.set_title(f"{name}: distribution of {focus}")
        ax.set_xlabel(focus)
        ax.set_ylabel("patients")
```

Read it as: **one `hist` call per target group, drawn on the same axes.**

- `df.groupby(target)` splits the frame into one sub-frame per value of the target column. Practical 6 territory.
- `for label, group in ...` unpacks each `(name, sub-frame)` pair.
- `group[focus].dropna()` picks one column out of the sub-frame and drops missing values. The `.dropna()` is not decoration — a NaN in a histogram is not skipped, it poisons the bin range, and the whole chart comes out empty.
- `bins=30` splits the column's range into 30 equal-width buckets and counts how many values land in each. 30 is a readable default for 768 rows; with 10 bins you see lumps, with 200 you see noise.
- `label=f"{target}={label}"` names the series. Without it `ax.legend()` prints "series1", "series2".

Why `alpha=0.6`. The two groups' histograms occupy the same x-range and overlap heavily — 500 non-diabetic and 268 diabetic patients in the same dataset. Drawn at full opacity, whichever series is plotted second paints over the first and you simply cannot tell there were two. At 60% opacity both are visible where they overlap, because the blended colour is darker than either bar on its own. It is the standard matplotlib idiom for "these two things are drawn in the same place, so let both be seen".

You can check this on the real output: `static/charts/pima_diabetes_Pregnancies_hist.png` shows the blue (`Outcome=0`) and orange (`Outcome=1`) bars blending into a darker shade across the crowded low-value region.

`ax.legend()` with no arguments then draws the key. With no arguments matplotlib picks the location (`loc="best"`) — on the histogram it lands top-right, over empty space, because that distribution is left-skewed. That luck does not hold everywhere, which is section 5.

3.4 `ax.set_title` / `ax.set_xlabel` / `ax.set_ylabel`

```
        ax.set_title(f"{name}: distribution of {focus}")
        ax.set_xlabel(focus)
        ax.set_ylabel("patients")
```

Three setters, one each for the chart title, the horizontal axis label and the vertical axis label. All three are called on `ax`, never on `plt`, for the ownership reason in 3.2.

The axis labels are not decoration either. `ax.set_ylabel("patients")` is the fix for a genuine ambiguity: `ax.hist` counts observations, and for a column like `Glucose` the count of glucose readings is meaningless without saying what is being counted. `ax.set_xlabel(focus)` echoes the column name so the PNG is self-describing once it leaves the API — the frontend shows it in an `<img>` tag with no surrounding context.

Every string is an f-string built from `name` and `focus`, so a chart cannot exist without saying which dataset and which column it came from.

3.5 The reference line

```
        ax.axhline(1.0, color="#888", linewidth=0.8, linestyle="--")
```

`axhline` draws a horizontal line across the whole axes at a given y value. In this chart y means "fraction of the overall mean" (section 4), so `1.0` is by definition the average — every bar above the line is above average for that column, every bar below is below.

```
        ax.set_ylabel("x average")
```

The label names that convention out loud. Without the dashed line, five bars sitting at 0.81 and 1.19 is a number; with it, they are "below average" and "above average", which is the only thing a reader wants to know. The styling is deliberately quiet: `#888` is mid-grey (matplotlib colours are hex strings *without* a `#`-prefixed CSS name), `linewidth=0.8` is thinner than the default 1.0, and `linestyle="--"` is dashed — all three say "reference, not data", so the line never competes with the bars for attention.

3.6 `fig.savefig(...)` and `plt.close(fig)`

```
        path = os.path.join(CHART_DIR, f"{slug}_{focus}_hist.png")
        fig.savefig(path, dpi=100, bbox_inches="tight")
        plt.close(fig)
        written.append(path)
```

`fig.savefig(path, dpi=100, bbox_inches="tight")`. Note it is `fig.savefig`, not `plt.savefig` — the figure you were given is the one that gets written. Three arguments:

- `path` — from `os.path.join`, Practical 4. The filename is built from a `slug` (`name.replace(".csv", "")`, back in section 3 of `save_charts`) plus the column name, so two datasets cannot collide and re-

running overwrites rather than accumulating near-duplicates.

- `dpi=100` — dots per inch. Combined with `figsize=(7, 4)` this yields a 700×400 image. 100 is matplotlib's default; it is written out because a chart that looks right on one screen and blurry on another is almost always an unset or mis-set dpi.
- `bbox_inches="tight"` — crop the surrounding whitespace. Without it, matplotlib pads every figure with a fixed margin that ignores the actual size of the text, so a chart with a long title wastes a third of its height. `"tight"` computes the real bounding box of everything drawn and trims to it. This is why the saved PNGs have no dead border.

`plt.close(fig)` is not cleanup, it is correctness. matplotlib keeps every figure it creates in a global registry of open figures, and it does **not** garbage collect them. A short script that draws three charts and exits leaks three figures, which nobody notices. A long-running process that draws one chart per request does not exit — and leaks one figure per request, forever, each one holding its full in-memory image buffer.

This project is exactly that shape of program. `save_charts` is called by a Flask route (`dataset_charts`) that can be hit repeatedly, and by `tests/test_analytics.py`, which calls it once per passing dataset. Over a long demo session the accumulated figures would grow until the process ran out of memory, with no error and no obvious cause. Three `plt.close(fig)` calls, one after each `savefig`, and the leak is gone.

The general rule, which is worth remembering beyond this project: **every figure you create in a long-running program must be closed.** If you draw in a loop, close in the loop.

3.7 Why `ax.hist()` and not `df.hist()`

pandas can plot. `df.hist()` draws a histogram of every numeric column at once, and `df.plot()` draws a line or bar chart, and both are one line instead of eight. This project refuses both, and says so in the docstring:

```
def save_charts(name):
    """Render charts to static/charts and return the filenames written.

    Uses explicit plt calls rather than df.hist() or df.plot() so that
    every parameter can be explained.

    The group means chart normalises each column to its own mean before
    plotting. Without that, Glucose at 0-140 flattens BMI at 0-35 into an
    invisible sliver, and the chart shows scale rather than signal.
    """
```

The reason is pedagogical, and it is the honest kind of reason. This is a lab submission, and `df.hist()` is a black box: it silently picks a bin count, silently picks a figure size, silently draws one subplot per column, and gives you no way to set `alpha` or a label. If a marker asks "how did you make this chart", the answer has to be a line of code, not a guarantee about a library's defaults.

Three concrete reasons the shortcuts would have failed here:

1. **`df.hist()` plots every column.** `pima_diabetes.csv` has eight numeric columns. The chart that matters is *one* column — the one `strongest_separators` picked — split by target. `df.hist()` gives no way to express "one column, grouped".
2. **Grouping is the entire point.** The split by target is the finding. Neither `df.hist()` nor `df.plot()` splits by a column value.
3. **The bugs in sections 4 and 5 were invisible until the call was explicit.** Both were found by rendering the chart and looking at it — which you can only do if you know what the chart is supposed to look like, and you only know that if you built it yourself.

One honest exception. The bar chart in section 4 *does* use pandas plotting:

```
        relative.plot(kind="bar", ax=ax)
```

That is a pandas call. The `ax=ax` keyword hands pandas the exact axes object to draw on, which is the OO-style discipline again — it is matplotlib underneath, just invoked through a pandas convenience wrapper. So the rule "explicit, not `df.plot()`" holds; the rule "never touch pandas plotting" does not, and pretending otherwise would be a small lie.

3.8 The `groupby().mean()` that feeds the bar chart

```
    # Bar chart: each column as a fraction of its own mean, by target.
    # Dividing by the column mean puts glucose, insulin and BMI on one
    # readable scale, and picks the columns that actually separate.
    if target is not None and cols:
        chosen = [s["column"] for s in separators[:5]] or cols[:5]
        means = df.groupby(target)[chosen].mean()
        relative = means / means.mean()

        fig, ax = plt.subplots(figsize=(9, 4.5))
        relative.plot(kind="bar", ax=ax)
```

Three Practical 6 lines doing all the analytical work:

- **`[s["column"] for s in separators[:5]]`** — take the top five column names out of `strongest_separators`, the function that ranks numeric columns by how differently the two target groups spread. The `or cols[:5]` is a fallback: if the ranking came back empty, use the first five numeric columns instead of drawing nothing.
- **`df.groupby(target)[chosen].mean()`** — one row per target value, one column per measurement, each cell the average for that combination. This is the classic pandas aggregation shape, and the identical call appears in `strongest_separators` at `analytics.py:221`.
- **`relative = means / means.mean()`** — the fix, and section 4.

`means.mean()` is worth pausing on, because it is the least obvious line in the function. `means` is a DataFrame with two rows (target values 0 and 1) and five columns. `means.mean()` with no arguments returns a **Series of five values** — the average of each column across the two rows. Dividing a DataFrame by a Series divides *column by column*, matching on the column label, not position. So each of the five columns is divided by its own average.

That is the whole trick, and section 4 is what happens when you skip it.

---

4. Real bug one: the scale bug

This project got this wrong first, and it was only caught by rendering the chart and looking at it. Nothing failed. No exception, no test failure, no warning. The PNG was written, `save_charts` returned a filename, `test_save_charts_writes_pngs` passed, and the chart was close to useless.

What the first version did. It plotted the raw group means, and this block is the **superseded** code — kept here because the bug is the lesson. It does not appear in `analytics.py` today; compare it with 3.8, which is what shipped:

```
        means = df.groupby(target)[chosen].mean()

        fig, ax = plt.subplots(figsize=(9, 4.5))
        means.plot(kind="bar", ax=ax)
```

Why that is wrong. Those five columns do not share a unit or a range. The real numbers from `pima_diabetes.csv`, target column `Outcome`:

Column	Outcome=0 mean	Outcome=1 mean

Pregnancies	3.3	4.9
Insulin	68.8	100.3
Glucose	110.0	141.3
DiabetesPedigreeFunction	0.4	0.6
Age	31.2	37.1

Put those on one y-axis and matplotlib picks the axis range from the largest value — 141.3. Glucose towers. Insulin is nearly as tall. And then `DiabetesPedigreeFunction`, whose entire signal is a difference of **0.2**, and `Pregnancies`, whose signal is **1.6**, collapse into a fraction of a pixel above the baseline. Two of the five bars — the two most separating columns after Insulin — become invisible.

The chart showed **scale rather than signal**. Every column looked like "almost no difference", which is a completely different finding from "Pregnancies and DiabetesPedigreeFunction separate the groups as well as Glucose does". The chart did not fail; it lied.

This is worth dwelling on, because it is a whole category of bug. The code was correct. The arithmetic was correct. The output was a valid PNG of correct numbers that supported the opposite of the intended conclusion. **Only looking at the picture caught it.**

The fix.

```
means = df.groupby(target)[chosen].mean()
relative = means / means.mean()
```

One line. Divide each column by its own average, and every bar lands in the same neighbourhood. The same real numbers, normalised:

Column	Outcome=0	Outcome=1
Pregnancies	0.808	1.192
Insulin	0.813	1.187
Glucose	0.876	1.124
DiabetesPedigreeFunction	0.877	1.123
Age	0.914	1.086

Now every column is readable, and the *ranking* becomes visible: the groups differ most on Pregnancies and Insulin, and least on Age. That is the actual clinical finding, and the first version of the chart hid it.

The axis label changes to match — `ax.set_ylabel("x average")` — and the title becomes explicit about the unit: `"mean as a fraction of overall mean"`. Renaming the axis when you change what is plotted is not optional. A chart that silently switches units while keeping its old label is worse than no chart.

Two things the docstring gets slightly wrong, left in place on purpose. The `save_charts` docstring says "*Glucose at 0-140 flattens BMI at 0-35*". The measured values in this dataset are Glucose 0–199 and BMI 0–67.1, and BMI is not even among the five chosen columns for this dataset — the flattening culprit is really `DiabetesPedigreeFunction` at 0.4 versus 0.6. The docstring is an accurate description of the *problem* and a loose description of these particular numbers. It is quoted here verbatim rather than corrected, because the reasoning is the lesson and the arithmetic is a detail that drifts whenever the dataset changes.

The transferable rule. Before plotting columns of different units on one shared axis, normalise them — or plot them separately. A shared axis is a claim that the numbers are comparable, and that claim is false unless you have made it true.

---

5. Real bug two: the legend covered the bars

Also found by rendering and looking. Also fixed by one call.

What was wrong. The legend was drawn inside the plot area, over the data. `relative.plot(kind="bar", ax=ax)` labels each of the five columns as a series, and `ax.legend()` with no arguments picks a spot *inside* the axes. On a chart whose bars run from 0 to about 1.2 in two clusters, "inside" means on top of the bars. The chart was technically complete and practically unreadable — you could not tell which colour was Glucose.

The fix moves the legend out of the axes entirely and down below it:

```
        ax.legend(title=target, loc="upper center",
                  bbox_to_anchor=(0.5, -0.12), ncol=3, frameon=False)
```

Four arguments, each doing a specific job:

- **`title=target`** — names what the series are. The bars are columns and the clusters are target values, which is the opposite of the obvious reading; without the title the legend is actively misleading.
- **`loc="upper center"`** — which point of the legend box to pin to the anchor.
- **`bbox_to_anchor=(0.5, -0.12)`** — where to pin it, in axes coordinates. `(0.5, ...)` is horizontally centred; `-0.12` is 12% of the axes height *below* the bottom edge, because negative is downward in axes coordinates. That negative number is the entire fix: it moves the legend outside the plotting area where it cannot cover anything.
- **`ncol=3`** — three entries per row, so five labels take two rows instead of five. A five-row legend is tall enough to push the chart itself off the page.
- **`frameon=False`** — no opaque box behind the text. On a white page the frame is just a grey rectangle drawn around something that is already readable.

The result is visible in the real file `static/charts/pima_diabetes_groupmeans.png`: the dashed `1.0` line runs unobstructed across the full plot area, all ten bars are visible, and the legend sits in clear white space beneath the x tick labels.

`bbox_inches="tight"` earns its place here. Because the legend now extends below the figure, matplotlib's default fixed padding would clip it. `"tight"` recomputes the bounding box from everything actually drawn, legend included, so the saved PNG contains the whole thing. The two fixes are related: moving the legend down would have truncated it without `"tight"`.

The transferable rule. A legend placed by default sits inside the data and you must look at the result to know whether it collides. On the histogram in section 3.3 `ax.legend()` happens to land in empty space; on the bar chart it lands on the bars. Same call, different outcome. When a legend matters, place it deliberately.

---

6. `unreachable_bands`: when an empty bar is a lie

The third chart plots risk bands, and it carries a piece of logic that has nothing to do with matplotlib at all.

```
def unreachable_bands(series, thresholds):
    """Return band names the column's range can never enter.

    A band no row can reach is misleading in a chart, because an empty bar
    looks like "nobody in this group" rather than "this group cannot
    exist". Glucose tops out at 199, so a cut point of 200 leaves the high
    band permanently empty.
    """
```

The cut points live in `RISK_THRESHOLDS` (`analytics.py:43-59`), and for pima Glucose they are `[100, 140, 200]` — a fourth band, `high`, starts at 200. But Glucose in this dataset **maxes out at 199**:

```
min/max Glucose: 0 199   mean: 120.9
```

So the `high` band is not empty by chance. No row in this dataset can ever enter it. `risk_bands` only returns bands with a non-zero count:

```
        "bands": {k: bands.get(k, 0) for k in order if bands.get(k, 0) > 0},
```

so `high` is simply absent from the data. A bar chart of what comes back therefore draws three bars — `low`, `moderate`, `elevated` — and a reader has no way to know a fourth was defined. The natural, entirely wrong conclusion is "nobody in this cohort has dangerously high glucose." The truth is "no value in this dataset is high enough to be counted," which is a fact about the *instrument's* range, not about the patients.

`unreachable_bands` compares the column's actual min and max against the cut points and returns the band names that cannot be reached:

```
    smallest = float(values.min())
    largest = float(values.max())

    unreachable = []

    if smallest >= thresholds[2]:
        unreachable.append("low")
        unreachable.append("moderate")
        unreachable.append("elevated")
    elif smallest >= thresholds[1]:
        unreachable.append("low")
        unreachable.append("moderate")
    elif smallest >= thresholds[0]:
        unreachable.append("low")

    if largest < thresholds[0]:
        unreachable.append("moderate")
        unreachable.append("elevated")
        unreachable.append("high")
    elif largest < thresholds[1]:
        unreachable.append("elevated")
        unreachable.append("high")
    elif largest < thresholds[2]:
        unreachable.append("high")

    return unreachable
```

Two independent `if`/`elif` chains — one for the bottom of the range, one for the top — so a column can report unreachable bands at both ends.

And then the chart says so, by overwriting the title:

```
        if bands["unreachable_bands"]:
            ax.set_title(f"{name}: {column} risk bands\n"
                         f"(no {', '.join(bands['unreachable_bands'])} band: "
                         f"outside the data range)")
```

Note it calls `ax.set_title` **again**, replacing the title set four lines earlier. In matplotlib a second `set_title` overwrites the first rather than adding to it. The `\n` puts the caveat on its own line so the real title stays readable.

The output, verified by rendering it:

```
pima_diabetes.csv: Glucose risk bands
(no high band: outside the data range)
low 197   moderate 374   elevated 197
```

Three bars, and a title that says why there are only three.

One honest caveat about which chart you will actually see. `save_charts` plots only the *first* column that has thresholds defined:

```
    # Risk band chart for the first defined column.
    for column in available_risk_bands(name):
```

and `available_risk_bands` returns `sorted(...)` keys, which for `pima_diabetes.csv` is `['Age', 'BMI', 'Glucose']`. `Age` comes first, its cut points `[24, 29, 41]` are all reachable, and the loop `break`s. So the PNG on disk — `static/charts/pima_diabetes_Age_bands.png` — shows four bars and **no** caveat. The annotation code is real and does fire; it just is not what this particular file shows. The behaviour is pinned by `tests/test_analytics.py:326-329`, which walks every thresholded column of `parkinsons.csv` and checks the unreachable-band result.

The transferable rule is about honesty in output, not about matplotlib: **distinguish "zero" from "impossible".** A missing bar has at least two causes — nobody qualified, or the scale cannot express qualifying — and they demand opposite responses from whoever reads the chart. Say which one it is.

---

7. Questions this should answer

- Why must `matplotlib.use("Agg")` be called before `import matplotlib.pyplot`, and what happens if a Flask server omits it?
- What is the difference between a figure and an axes, and why does drawing on `ax` beat drawing on `plt` when one function builds three different charts?
- Why does `alpha=0.6` matter for a histogram of two overlapping groups, and why is `dropna()` not optional before calling `ax.hist()`?
- The group-means chart was correct code producing a misleading picture. What did `means / means.mean()` fix, and what does the resulting y-axis now measure?
- Why must every `fig` be closed with `plt.close(fig)` in a program that serves charts over HTTP, and what happens without it?
- An empty bar in the risk-band chart could mean "nobody qualified" or "nobody could qualify". What does `unreachable_bands` do to tell those apart, and how does that reach the chart's title?